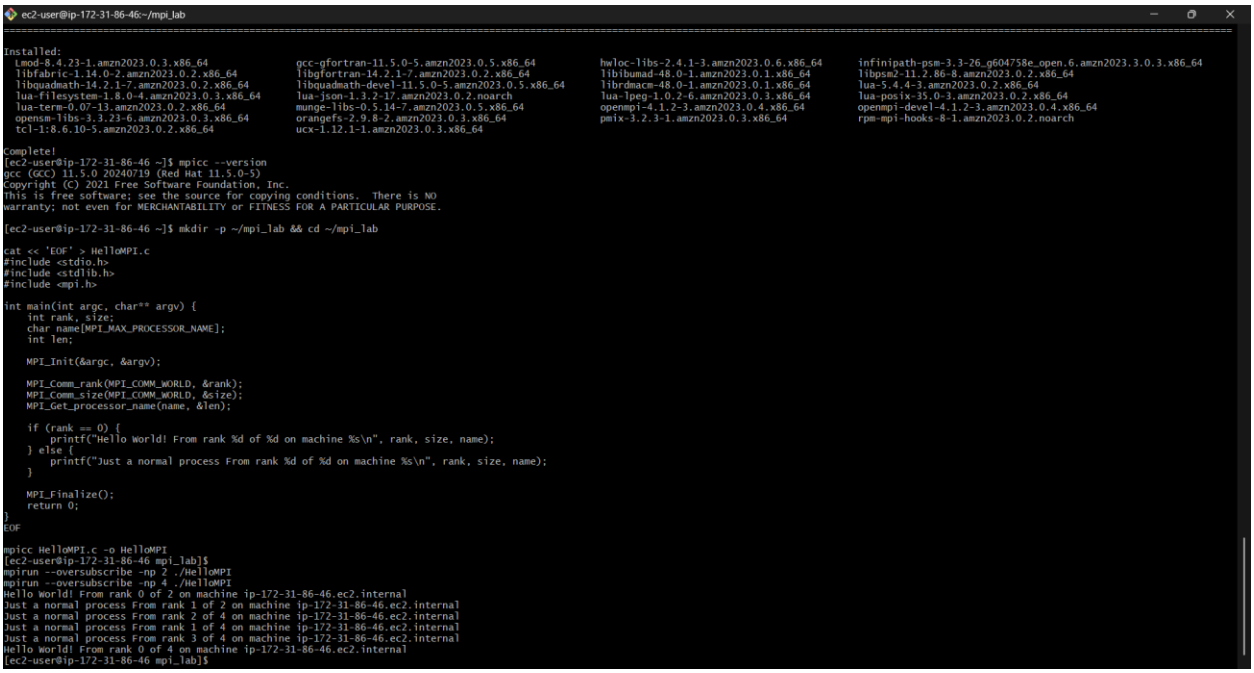
SE3082 - Parallel Computing

MPI-Part1

IT24610782 – M.A.V.E. MUNASINGHE



```
ec2-user@ip-172-31-86-46:~/mpi_lab
================================================================================
Installed:
  lmod-8.4.23-1.amzn2023.0.3.x86_64              gcc-gfortran-11.5.0-5.amzn2023.0.5.x86_64      hwloc-libs-2.4.1-3.amzn2023.0.6.x86_64      infinipath-psm-3.3-26_g604758e_open.6.amzn2023.3.0.3.x86_64
  libfabric-1.14.0-2.amzn2023.0.2.x86_64         libgfortran-14.2.1-7.amzn2023.0.2.x86_64       libibumad-48.0-1.amzn2023.0.1.x86_64        libpsm2-11.2.86-8.amzn2023.0.2.x86_64
  libquadmath-14.2.1-7.amzn2023.0.2.x86_64       libquadmath-devel-11.5.0-5.amzn2023.0.5.x86_64 librdmacm-48.0-1.amzn2023.0.1.x86_64       lua-5.4.4-3.amzn2023.0.2.x86_64
  lua-filesystem-1.8.0-4.amzn2023.0.3.x86_64     lua-json-1.3.2-17.amzn2023.0.2.noarch          lua-lpeg-1.0.2-6.amzn2023.0.3.x86_64        lua-posix-35.0-3.amzn2023.0.2.x86_64
  lua-term-0.07-13.amzn2023.0.2.x86_64           munge-libs-0.5.14-7.amzn2023.0.5.x86_64        openmpi-4.1.2-3.amzn2023.0.4.x86_64         openmpi-devel-4.1.2-3.amzn2023.0.4.x86_64
  opensm-libs-3.3.23-6.amzn2023.0.3.x86_64       orangefs-2.9.8-2.amzn2023.0.3.x86_64           pmix-3.2.3-1.amzn2023.0.3.x86_64            rpm-mpi-hooks-8-1.amzn2023.0.2.noarch
  tcl-1:8.6.10-5.amzn2023.0.2.x86_64             ucx-1.12.1-1.amzn2023.0.3.x86_64

Complete!
[ec2-user@ip-172-31-86-46 ~]$ mpicc --version
gcc (GCC) 11.5.0 20240719 (Red Hat 11.5.0-5)
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

[ec2-user@ip-172-31-86-46 ~]$ mkdir -p ~/mpi_lab && cd ~/mpi_lab

cat << 'EOF' > HelloMPI.c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    char name[MPI_MAX_PROCESSOR_NAME];
    int len;

    MPI_Init(&argc, &argv);

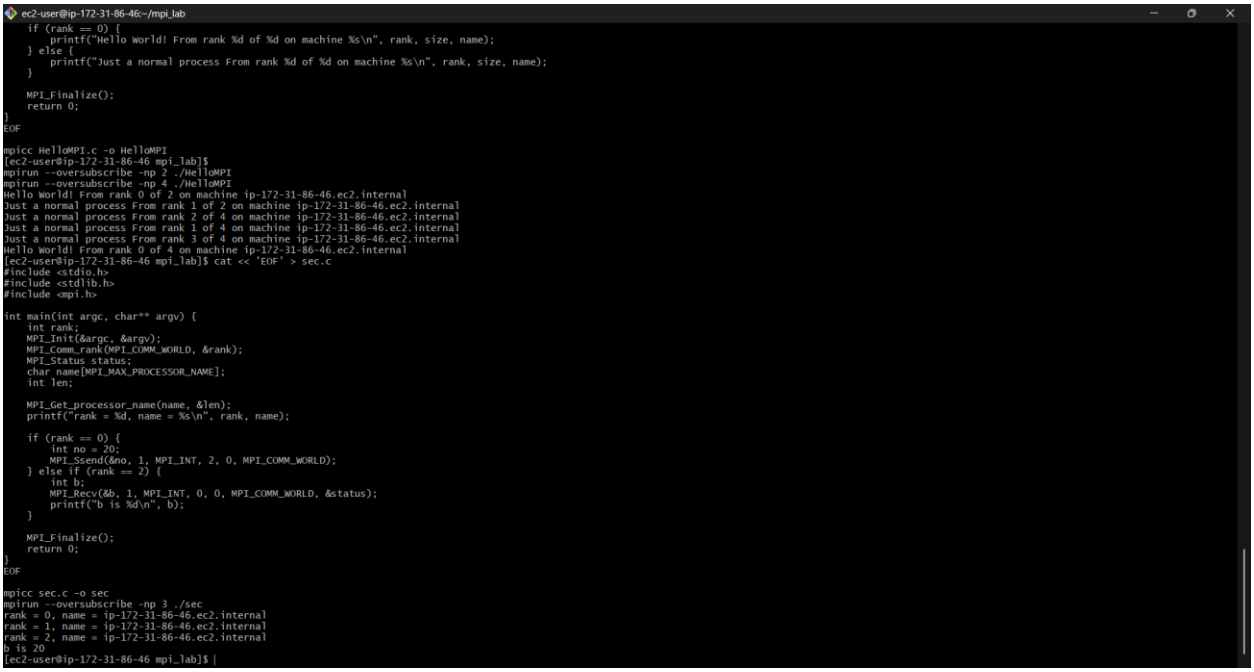
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(name, &len);

    if (rank == 0) {
        printf("Hello World! From rank %d of %d on machine %s\n", rank, size, name);
    } else {
        printf("Just a normal process From rank %d of %d on machine %s\n", rank, size, name);
    }

    MPI_Finalize();
    return 0;
}
EOF

mpicc HelloMPI.c -o HelloMPI
[ec2-user@ip-172-31-86-46 mpi_lab]$
mpirun --oversubscribe -np 2 ./HelloMPI
mpirun --oversubscribe -np 4 ./HelloMPI
Hello World! From rank 0 of 2 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 1 of 2 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 2 of 4 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 1 of 4 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 3 of 4 on machine ip-172-31-86-46.ec2.internal
Hello World! From rank 0 of 4 on machine ip-172-31-86-46.ec2.internal
[ec2-user@ip-172-31-86-46 mpi_lab]$
```

Sample C program with SSend and Receive



```
ec2-user@ip-172-31-86-46:~/mpi_lab
    if (rank == 0) {
        printf("Hello World! From rank %d of %d on machine %s\n", rank, size, name);
    } else {
        printf("Just a normal process From rank %d of %d on machine %s\n", rank, size, name);
    }

    MPI_Finalize();
    return 0;
}
EOF

mpicc HelloMPI.c -o HelloMPI
[ec2-user@ip-172-31-86-46 mpi_lab]$
mpirun --oversubscribe -np 2 ./HelloMPI
mpirun --oversubscribe -np 4 ./HelloMPI
Hello World! From rank 0 of 2 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 1 of 2 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 2 of 4 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 1 of 4 on machine ip-172-31-86-46.ec2.internal
Just a normal process From rank 3 of 4 on machine ip-172-31-86-46.ec2.internal
Hello World! From rank 0 of 4 on machine ip-172-31-86-46.ec2.internal
[ec2-user@ip-172-31-86-46 mpi_lab]$ cat << 'EOF' > sec.c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Status status;
    char name[MPI_MAX_PROCESSOR_NAME];
    int len;

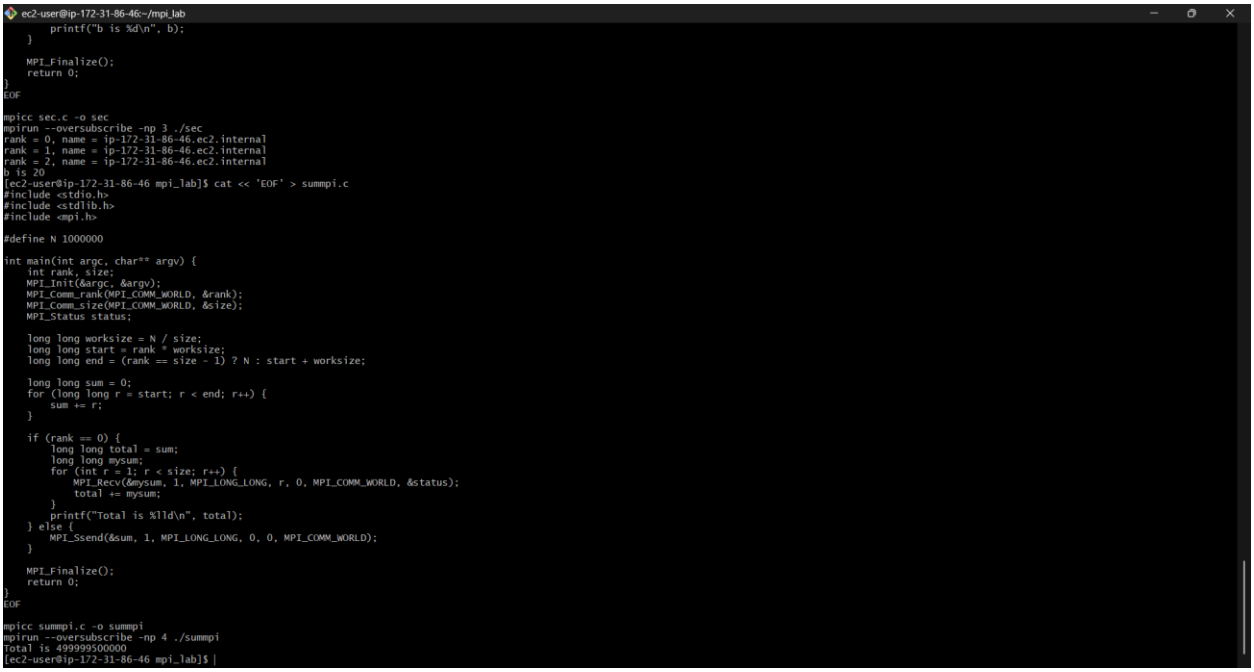
    MPI_Get_processor_name(name, &len);
    printf("rank = %d, name = %s\n", rank, name);

    if (rank == 0) {
        int no = 20;
        MPI_Ssend(&no, 1, MPI_INT, 2, 0, MPI_COMM_WORLD);
    } else if (rank == 2) {
        int b;
        MPI_Recv(&b, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        printf("b is %d\n", b);
    }

    MPI_Finalize();
    return 0;
}
EOF

mpicc sec.c -o sec
mpirun --oversubscribe -np 3 ./sec
rank = 0, name = ip-172-31-86-46.ec2.internal
rank = 1, name = ip-172-31-86-46.ec2.internal
rank = 2, name = ip-172-31-86-46.ec2.internal
b is 20
[ec2-user@ip-172-31-86-46 mpi_lab]$
```

Calculating numbers upto 1million



Counting to 1 million - Using Rank for breaking work across nodes

